ICS 214 Lab 5

Objectives:

1. Use pointers to create, view, and delete entries in a linked list.
2. Use dynamic memory allocation and de-allocation for each node in the list.
3. Add const to function parameters when appropriate.

Intro

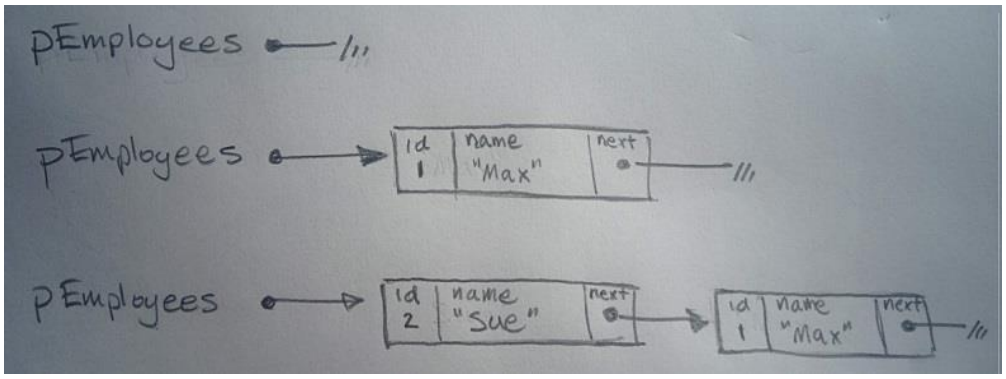
This lab is a bit different from the others. You will be adding to existing code to create some basic functionality for the following menu (which has already been implemented).

```
--- MENU ---
1) View Employees
2) Add Employee
3) Remove Employee
0) Exit
-------------
Select:
```

The existing code already takes care of displaying the menu, receiving input, and calling functions that are commented with descriptions of what they should do. Your task is to fill in the empty functions to implement a linked list of employee records using dynamic memory allocation.

The functions you complete will provide functionality for a linked list of Employee structs. The list will be tracked with a local pointer in the main() function (Employee* pEmployees). No globals are to be used. This pointer, which represents the first node in the list, is passed to functions that need to access/modify it. The functions that receive this pointer refer to this pointer as pHead (the head or start of the list).

Each Employee node contains an id, a name, and an Employee* pointer (next) which points to the next node (if there is one). Each node is an instance of the Employee struct which has been dynamically allocated via createEmployee(). Here's a graphic view of the expected implementation of the linked list…



This diagram shows the linked list:

- At the start (where pEmployees is nullptr)
- after adding "Max" as an employee
- after adding "Sue" as an employee (new nodes are added to the front of the list)

Member selection using operator->

You will be dealing with pointers to structs in this lab. We've learned that with normal variables, we can access objects directly. However, pointers hold addresses, so we first need to dereference the pointer to get the object before we can access it. We've seen the syntax for how to do this in class (line 17 below). C++ also provides the "arrow operator" (also known as operator->) to make the syntax dereferencing a pointer to an object easier (line 21 below).

```
6   struct Fraction {
7       int numerator;
8       int denominator;
9   };
10
11  int main() {
12      // create a fraction on the heap, assign the address to a pointer
13      Fraction* pFraction = new Fraction{ 2,6 };
14
15      // To access members, dereference the pointer first, then use member
16      // selection ("." operator).  This works fine, but syntax is complex.
17      std::cout << (*pFraction).numerator;
18
19      // Cleaner syntax using the "arrow operator".
20      // Accomplishes the exact same thing.
21      std::cout << pFraction->numerator;
22  }
```

More details on this can be found on Learncpp.com in section 13.12 – Member selection with pointers and references. As best practice, they recommend using operator-> instead of the dot operator (for member selection using pointers). You can make use of operator-> throughout this lab.

Passing a pointer by reference

There will be a function declaration given to you that that contains a bit of syntax that you may be unsure about:

```
void addNewEmployee(Employee*& pHead, const std::string& fName);
```

This syntax (*&) looks a bit daunting, but really, it's just a combination of two concepts we already know: passing a pointer, and pass-by-reference.

This intent of this function is to add a new node to the start of the linked list. The first parameter (pHead) is a pointer to the memory address of the first Employee struct in a linked list. We want the argument passed into the function to point to the new node. To affect the original argument passed to the parameter pHead, we need to pass by reference.

> *Further explanation: Without the '&' we are passing a memory address "by value". This would make the parameter pHead a new local variable, and the argument (a memory address) passed in would be copied into it. The implementation of the function would tell pHead to point to a new node (allocated dynamically). At the end*

of the function, pHead would fall out of scope and die (since it is a local var). The dynamically allocated node it points to would be lost (resulting in a memory leak). The original argument from the caller (a pointer used to track the front of the list) was never actually changed since the pointer was passed by value – only pHead (a local copy) was.

How do we fix this? We pass the pointer by reference so the variable in the calling function (the argument passed in) is affected.

`Employee*& pHead` - is just a pointer that is passed by reference which allows the original pointer to be modified (the same as passing any other variable by reference).

Hints:

1) Check pointers for null before dereferencing.

* If a function uses a pointer passed in as a parameter, you need to check if it is null before dereferencing it. A dereferenced null pointer causes undefined behaviour (and likely a crash)! Ensure that before you dereference a pointer, you are certain it is non-null.

```
void sampleFunction(Employee* pHead, int id) {

	std::cout << pHead->id;			// this could crash if pHead is null
	if (pHead) { std::cout << pHead->id; }	// safe

}
```

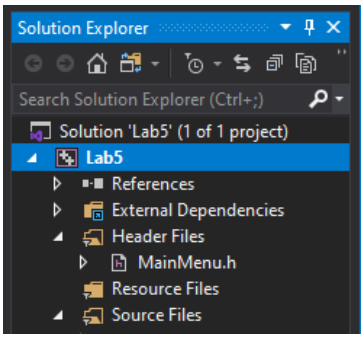
2) Draw what you want to happen with pointers on paper.

When using pointers, it is sometimes difficult to picture exactly what is going on. It is also difficult to view the memory because we have to do so via pointers. If you're having difficulty writing any of the functions, **draw it out**. See the diagram on the first page for an example. Draw what the state of the linked list is before the function runs, at each step, and the expected state after. Make sure what you are doing in code reflects what you want to happen visually. With pointers it is nearly impossible to just guess and get it right. You have to know exactly what you are doing!

A Dynamically Allocated Linked List

1. Create a new empty project called Lab5.

2. Download & extract **Lab5Source.zip** from D2L. Add the contents to your project: Main.cpp, MainMenu.cpp, MainMenu.h, TestSuite.h, TestSuite.cpp
   To do this, right click on the project name in your solution explorer, and then select "Open folder in File Explorer". Copy and past the files into the folder that opens. Next, in Visual Studio, right click on source files (in the Solution Explorer). Select Add | Existing Item, and then select all the files that you just added. Click Add to add them to the project. You can then drag and drop the .h files (in the Solution Explorer) into the Header Files directory. What you are seeing in the Solution Explorer are "virtual links" so it will not change the physical location of the file – it only changes the view in the Solution Explorer.



3. Run the program, and then read through the existing code and comments to get a sense of the "current state" of the program. The code has been commented with "**TODO**" wherever there is a functionality that needs to be added by you.

4. Read through the page titled "Checking for Memory Leaks" below. Add the code specified to your project and prove to yourself that the memory check works. It will be helpful to be mindful of when you need to de-allocate memory as you program.

5. Fill out functions in the following order (for ease of testing). Follow the comments above each function regarding implementation.
   - **createEmployee()**
   - **addNewEmployee()**
   - **viewEmployees()**
   - **getNodeInfo()**
   - **removeEmployee()**

   Verify each function works (using the debugger or the console), even if it means calling the newly written function in main() all on its own. You can view the node pointers and the contents of them in the debugger to ensure things are happening the way you think they are before writing the next function.

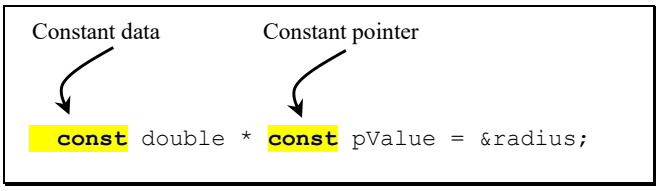
   Output for the viewEmployees operation should look like the example below:

```
--- MENU ---
1) View Employees
2) Add Employee
3) Remove Employee
4) Remove all
0) Exit
-------------
Select:1
>> View Employees:
[id: 1, name: Ben]
[id: 0, name: Amy]
```

6. Add a new function to clear the list called **removeAllEmployees()**. Provide it as an option on the menu.

7. Before the program exits, clean up by ensuring the list is destroyed and return all dynamically allocated memory to the heap. You can do this via removeAllEmployes(). Approximately 50% of students from the previous year forgot to do this.

8. Review each function and when appropriate, add "const" to any parameters that are not changed by the function. For pointers, it is possible to add "const" in 2 different places.



The first const makes the pointer a pointer to a const object and has the following effect:

- It prevents the block of memory the pointer points to from being changed.
- It assures the caller that no changes will be made to the argument.
- It allows arguments that are const to be passed to the function.

It is likely that you'll be using this whenever possible because of the associated benefits (much like passing by const reference).

The second const makes the pointer a const pointer. It prevents the pointer from being re-assigned to point to another block of memory. It is really only used if there is good reason for preventing a pointer re-assignment. It is likely that you won't need to use this much at all, but at least you know what it does!

Use a pointer to a const object when:

a. The function doesn't change the contents of the pointer, and
b. The program doesn't need to change anything returned by the function?

Use a const pointer if:

- There is an important reason for preventing the function from changing the address that the pointer points to?

9. OPTIONAL CHALLENGE (not for marks): For those looking for an additional challenge, modify addNewEmployee() to take a Boolean parameter called "alphabetical" which defaults to true. When alphabetical is false, use the current implementation (nodes are inserted at the front of the list). When alphabetical is true, assume the list is sorted, and insert the newest record in alphabetical order (of name). Make sure you currently don't have any memory leaks before doing this though (see next page)

Checking for memory leaks:

There are 2 ways I know of to test for memory leaks.

1. The first involves adding a few lines of code to your project. When your program stops, it will inform you if there was a memory leak and what the size is. The advantage: it is easy to add, and errors are output automatically.
2. The second involves using the Visual Studio runtime diagnostic tools (heap profiling) to take snapshots at different points in execution. The advantage is that snapshots can be taken at strategic points in code which allow you to narrow down where a memory leak is coming from.

Let's plan to use method 1 (as a minimum requirement). If you need more details about exactly when/how the leak occurs, you can experiment with method 2. These two methods are detailed below.

Method 1: Automated via code

There is a way to have your program test for memory leaks using the C run-time library (CRT) as described in the following article:

https://docs.microsoft.com/en-us/visualstudio/debugger/finding-memory-leaks-using-the-crt-library?view=vs-2019

Here are the highlights:

Step 1: Add this code to the top of the .cpp file with main() in it:

```
#define _CRTDBG_MAP_ALLOC
#include <stdlib.h>
#include <crtdbg.h>
```

Step 2: Add this code at the top of your main() function:

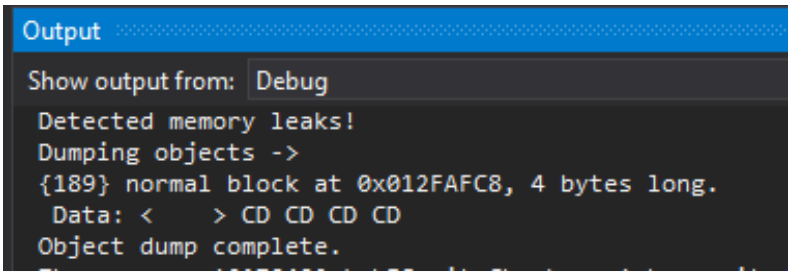
```
_CrtSetDbgFlag ( _CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF );
```

If you run your program and there are memory leaks, it will report them in the output window when you exit your program). Try adding the following to main() to illustrate the error you will see when you have a memory leak:

```
22      int* x = new int;   // allocate an integer on the heap
```

Now run your program and exit via 0 on the menu (memory leaks won't show if you stop your program with the stop button.

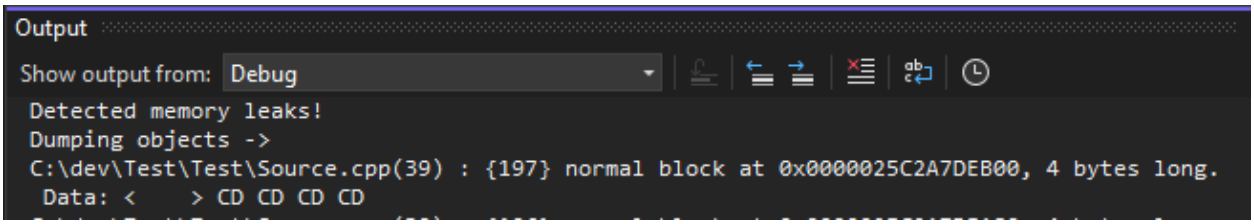
The following test run indicated a memory leak because an int (4 bytes) was allocated dynamically (on the heap) without deleting it (returning it to the heap). You can see that it reports 4 bytes have been leaked!



This is helpful, but it doesn't say anything about where the leak occurs. However, there is a way to get the line number of the allocation.

1. Remove the code added in step 1 (but leave step 2).
2. Add "MemoryDebug.h" to the project (if not present already). Open this .h file and have a look. The step 1 code was copied to this .h file. It adds some code that references a macro name that will tell use if we are in debug mode (_DEBUG), and it creates a new macro (DBG_NEW) that we can be used in our code to replace any occurrences of the "new" keyword with a special version of new if we are running in debug mode. Read the comments in the file to understand exactly how this works.
3. #include "MemoryDebug.h" at the top of the .cpp file that contains main() along with any .cpp files that allocate heap memory with the new keyword
4. Replace any code occurrences of the new keyword with DBG_NEW (it will be changed back by the preprocessor)

If you run in debug mode now, you should get output that tells you the line number of an allocation that caused a memory leak (in the example below: line 39 of Source.cpp):



Once you are satisfied this works, remove the dynamic allocation that caused this problem.

Method 2: Visual Studio snapshots

We will look at how to use this method in the lab or class time. If you miss it, details on how to use the heap profiler can be found in the following article: https://learn.microsoft.com/en-us/visualstudio/profiling/memory-usage?view=vs-2022.

It is new approach (for me), but I wanted to introduce it as an option because it seemed straight-forward, specific, and I found that it gave me more insight than using method 1 (on a larger project). Also, method 1 only reported leak information once the program stopped (making it more difficult to narrow down where a leak is occurring). Snapshots can be used at runtime to help isolate where a memory leak occurs.

Testing & demo preparation:

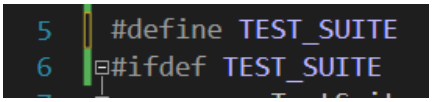
Before you demo, test all the edge cases you can think of:

- View an empty list.
- Add a single element. Add 5 nodes (a,b,c,d,e). Removing the last, middle, and first nodes and verify that at each step of the way the list looks as it should.
- Remove the remaining nodes one by one. View.
- Remove an invalid id with an empty list
- Remove an invalid id with a list of one element

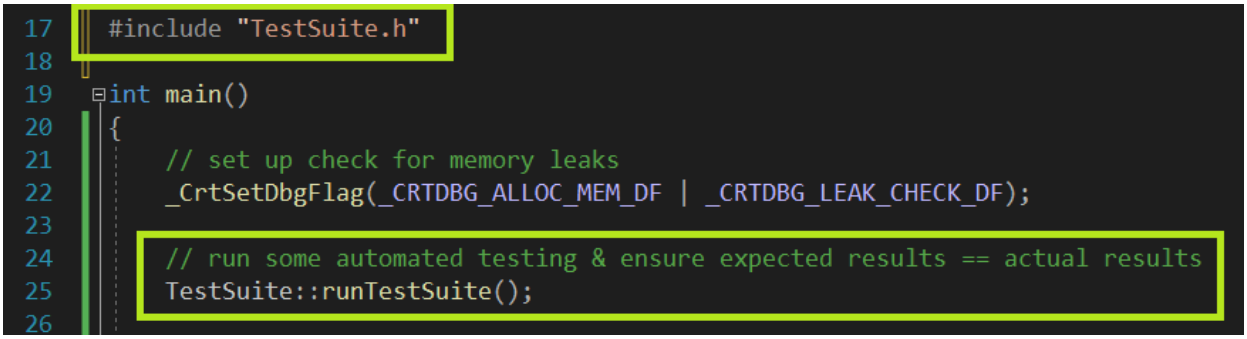
Automated testing:

There is a suite of automated tests to run for your demo:

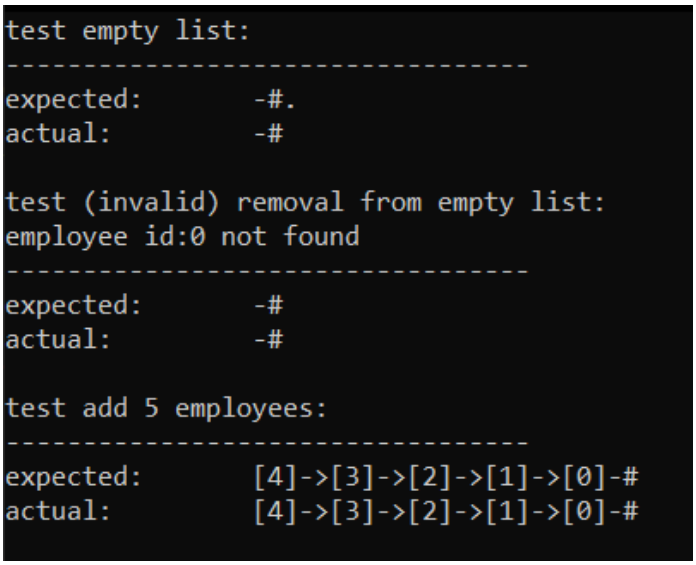
1. #include “TestSuite.h” at the top of Main.cpp
2. Uncomment line 5 in TestSuite.cpp to include TestSuite code in the compilation.



3. Add the following in main()



This will run some automated testing for your linked list and output expected and actual results to the console:



Ensure that all expected results are the same as the actual results. If they don't match, then likely something is wrong.

Code Review

When you are finished this assignment, find a student who is also finished. Conduct a code review on each other's code **before** giving a demo of your assignment for marks.

Marking:

- Does it work as expected?
- Variables: Have good variable & const names been used?
- Are variables in local scope instead of global?
- Are there comments to explain anything non-trivial or not obvious
- Have variables been initialized with best practices (uniform initialization).

[1/10] createEmployee()

[1/10] addNewEmployee()

[1/10] viewEmployees()

[2/10] getNodeInfo()

[2/10] removeEmployee()

[1/10] removeAllEmployees()

[2/10] no memory leaks from start to finish

Due dates:

You have one week to complete this lab. It will be due a week from your regular lab period that it is assigned in. The week it is assigned in can be seen on D2L in Content |Start Here |Class/Lab schedule.